

**Міністерство освіти і науки України Національний технічний університет
України «Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки Кафедра
обчислювальної техніки**

Лабораторна робота №6

з дисципліни
«Алгоритми і структури даних»

Виконала:

студентка групи ІМ-44

Крошка Дар'я Олександрівна номер у
списку групи: 11

Перевірив:

Сергієнко А. М.

Київ 2025

Постановка задачі

1. Представити зважений ненапрямлений граф із заданими параметрами так само, як у лабораторній роботі №3.

Відмінність 1: коефіцієнт $k = 1.0 \cdot n^3 \cdot 0.01 \cdot n^4 \cdot 0.005 \cdot 0.05$.

Отже, матриця суміжності A_{dir} напрямленого графа за варіантом формується таким чином:

- 1) встановлюється параметр (seed) генератора випадкових чисел, рівне номеру варіанту $n_1 n_2 n_3 n_4$;
- 2) матриця розміром $n \times n$ заповнюється згенерованими випадковими числами в діапазоні $[0, 2.0)$;
- 3) обчислюється коефіцієнт $k = 1.0 \cdot n^3 \cdot 0.01 \cdot n^4 \cdot 0.005 \cdot 0.05$, кожен елемент матриці множиться на коефіцієнт k ;
- 4) елементи матриці округлюються: 0 — якщо елемент менший за 1.0, 1 - якщо елемент більший або дорівнює 1.0.

Матриця A_{undir} ненапрямленого графа одержується з матриці A_{dir} так само, як у ЛР №3. Відмінність 2: матриця ваг W формується таким чином.

- 1) матриця B розміром $n \times n$ заповнюється згенерованими випадковими числами в діапазоні $[0, 2.0)$ (параметр генератора випадкових чисел той же самий, $n_1 n_2 n_3 n_4$);
- 2) одержується матриця C : $c_{ij} = \text{ceil}(pb_{ij} \cdot 100 \cdot a_{undir,i,j} \cdot q)$, $c_{i,j} \in C$, $b_{ij} \in B$, $a_{undir,i,j} \in A_{undir}$, де ceil — це функція, що округляє кожен елемент матриці до найближчого цілого числа, більшого чи рівного за дане;
- 3) одержується матриця D , у якій
 $d_{ij} = 0$, якщо $c_{ij} = 0$,
 $d_{ij} = 1$, якщо $c_{ij} \neq 0$, $d_{ij} \in D$, $c_{ij} \in C$;
- 4) одержується матриця H , у якій
 $h_{ij} = 1$, якщо $d_{ij} = d_{ji}$,
та $h_{ij} = 0$ в іншому випадку;
- 5) Tr - верхня трикутна матриця з одиниць ($tr_{ij} = 1$ при $i = j$);

б) матриця ваг W симетрична, і її елементи одержуються за формулою: $w_{ij} = w_{ji}$
 $p_{dij} = h_{ij} - t_{rij} - q_{cij}$.

2. Створити програму для знаходження мінімального кістяка за алгоритмом Краскала при $n4$ - парному і за алгоритмом Пріма - при непарному. При цьому у програмі:

- графи представляти у вигляді динамічних списків, обхід графа, додавання, віднімання вершин, ребер виконувати як функції з вершинами відповідних списків;
- у програмі виконання обходу відображати покроково, черговий крок виконувати за натисканням кнопки у вікні або на клавіатурі.

3. Під час обходу графа побудувати дерево його кістяка. У програмі дерево кістяка виводити покроково у процесі виконання алгоритму. Це можна виконати одним із двох способів:

- або виділяти іншим кольором ребра графа;
- або будувати кістяк поряд із графом.

4. При зображенні як графа, так і його кістяка, вказати ваги ребер.

5. При проектуванні програми також слід врахувати наступне:

- 1) мова програмування обирається студентом самостійно;
- 2) графічне зображення усіх графів має формуватися програмою з тими ж вимогами, як у ЛР №3;
- 3) всі графи обов'язково зображувати у графічному вікні;
- 4) типи та структури даних для внутрішнього представлення всіх даних у програмі слід вибрати самостійно.

Программный код

```
import numpy as np
import tkinter as tk
from tkinter import messagebox
import math

n1, n2, n3, n4 = 4, 4, 1, 1
n = 10 + n3 # n = 11
seed = int(f"{n1}{n2}{n3}{n4}")
k = 1.0 - n3 * 0.01 - n4 * 0.005 - 0.05

print(f"Parameters: n1={n1}, n2={n2}, n3={n3}, n4={n4}")
print(f"n={n}, seed={seed}, k={k:.3f}")
print(f"Algorithm: Prim (n4={n4} is odd)")

np.random.seed(seed)

A_random = np.random.rand(n, n) * 2.0
A_dir = ((A_random * k) >= 1.0).astype(int)
np.fill_diagonal(A_dir, 0)

A_undir = np.logical_or(A_dir, A_dir.T).astype(int)

np.random.seed(seed)
B = np.random.rand(n, n) * 2.0

C = np.ceil(B * 100 * A_undir).astype(int)

D = (C != 0).astype(int)

H = (D == D.T).astype(int)

Tr = np.triu(np.ones((n, n)), k=1).astype(int)

W = np.zeros((n, n))
for i in range(n):
    for j in range(n):
        if D[i,j] * H[i,j] * Tr[i,j] == 1:
            W[i,j] = W[j,i] = C[i,j]

print(f"\nUndirected adjacency matrix Aundir:")
print(Aundir)
```

```
print(f"\nWeight matrix W:")
```

```
print(W)
```

```
class Graph:
```

```
    """Graph representation using adjacency lists"""
```

```
    def __init__(self, adjacency_matrix, weight_matrix):
```

```
        self.n = len(adjacency_matrix)
```

```
        self.adj_list = {}
```

```
        self.weights = {}
```

```
        for i in range(self.n):
```

```
            self.adj_list[i] = []
```

```
            for j in range(self.n):
```

```
                if adjacency_matrix[i][j] == 1 and weight_matrix[i][j] > 0:
```

```
                    self.adj_list[i].append(j)
```

```
                    self.weights[(i, j)] = weight_matrix[i][j]
```

```
                    self.weights[(j, i)] = weight_matrix[i][j]
```

```
    def get_neighbors(self, vertex):
```

```
        """Get neighbors of a vertex"""
```

```
        return self.adj_list.get(vertex, [])
```

```
    def get_weight(self, u, v):
```

```
        """Get weight of edge (u,v)"""
```

```
        return self.weights.get((u, v), 0)
```

```
    def get_all_edges(self):
```

```
        """Get all edges with weights"""
```

```
        edges = []
```

```
        seen = set()
```

```
        for u in range(self.n):
```

```
            for v in self.get_neighbors(u):
```

```
                if (u, v) not in seen and (v, u) not in seen:
```

```
                    edges.append((u, v, self.get_weight(u, v)))
```

```
                    seen.add((u, v))
```

```
        return edges
```

```
class PrimMSTApp:
```

```
    def __init__(self, root, graph, weight_matrix):
```

```
        self.root = root
```

```
        self.graph = graph
```

```
        self.weight_matrix = weight_matrix
```

```
        self.n = graph.n
```

```

self.mst_edges = []
self.visited = set()
self.priority_queue = []
self.total_weight = 0
self.step_count = 0
self.algorithm_finished = False

self.setup_ui()

self.positions = {
    i: (350 + 200 * np.cos(2 * np.pi * i / self.n),
        300 + 200 * np.sin(2 * np.pi * i / self.n))
    for i in range(self.n)
}

self.visited.add(0)
self.update_priority_queue()

self.draw_graph()
self.update_info()

def setup_ui(self):
    """Setup user interface"""
    self.canvas = tk.Canvas(self.root, width=700, height=600, bg="white")
    self.canvas.pack()

    button_frame = tk.Frame(self.root)
    button_frame.pack(pady=10)

    self.step_button = tk.Button(button_frame, text="Наступний крок",
                                command=self.prim_step, font=("Arial", 12))
    self.step_button.pack(side=tk.LEFT, padx=5)

    self.reset_button = tk.Button(button_frame, text="Скинути",
                                command=self.reset_algorithm, font=("Arial",
12))
    self.reset_button.pack(side=tk.LEFT, padx=5)

    self.info_text = tk.Text(self.root, height=15, width=90,
font=("Courier", 9))
    self.info_text.pack(pady=5)

```

```

        scrollbar = tk.Scrollbar(self.info_text)
        scrollbar.pack(side=tk.RIGHT, fill=tk.Y)
        self.info_text.config(yscrollcommand=scrollbar.set)
        scrollbar.config(command=self.info_text.yview)

def update_priority_queue(self):
    """Update priority queue with edges from visited vertices"""
    self.priority_queue = []

    for u in self.visited:
        for v in self.graph.get_neighbors(u):
            if v not in self.visited:
                weight = self.graph.get_weight(u, v)
                if weight > 0:
                    self.priority_queue.append((weight, u, v))

    self.priority_queue.sort()

def prim_step(self):
    """Perform one step of Prim's algorithm"""
    if self.algorithm_finished:
        return

    if not self.priority_queue:
        self.algorithm_finished = True
        messagebox.showinfo("Prim's Algorithm",
                             f"Алгоритм завершено!\nЗагальна вага MST:
{self.total_weight}")
        return

    weight, u, v = self.priority_queue.pop(0)

    if v in self.visited:
        self.prim_step()
        return

    self.mst_edges.append((u, v, weight))
    self.visited.add(v)
    self.total_weight += weight
    self.step_count += 1

    self.update_priority_queue()

```

```

self.draw_graph()
self.update_info()

if len(self.mst_edges) == self.n - 1:
    self.algorithm_finished = True
    messagebox.showinfo("Prim's Algorithm",
                        f"MST побудовано!\nЗагальна вага:
{self.total_weight}")

def reset_algorithm(self):
    """Reset the algorithm to initial state"""
    self.mst_edges = []
    self.visited = {0}
    self.priority_queue = []
    self.total_weight = 0
    self.step_count = 0
    self.algorithm_finished = False

    self.update_priority_queue()
    self.draw_graph()
    self.update_info()

def draw_graph(self):
    """Draw the graph with current MST state"""
    self.canvas.delete("all")

    for u in range(self.n):
        for v in self.graph.get_neighbors(u):
            if u < v:
                weight = self.graph.get_weight(u, v)
                if weight > 0:
                    x1, y1 = self.positions[u]
                    x2, y2 = self.positions[v]

                    is_mst_edge = any((min(u,v), max(u,v)) == (min(eu,ev),
max(eu,ev))

                                for eu, ev, ew in self.mst_edges)

                    if is_mst_edge:
                        color = "red"
                        width = 4
                    else:
                        color = "gray"

```



```

        width = 1

        self.canvas.create_line(x1, y1, x2, y2, fill=color,
width=width)

        mid_x, mid_y = (x1 + x2) / 2, (y1 + y2) / 2
        self.canvas.create_oval(mid_x-12, mid_y-8, mid_x+12,
mid_y+8,

                                fill="white", outline="black")
        self.canvas.create_text(mid_x, mid_y,
text=str(int(weight)),

                                font=("Arial", 8, "bold"))

    for i in range(self.n):
        x, y = self.positions[i]

        if i in self.visited:
            color = "lightgreen"
        else:
            color = "lightblue"

        self.canvas.create_oval(x-20, y-20, x+20, y+20,
                                fill=color, outline="black", width=2)
        self.canvas.create_text(x, y, text=str(i+1), font=("Arial", 12,
"bold"))

    def update_info(self):
        """Update information display"""
        self.info_text.delete(1.0, tk.END)

        info = f"Алгоритм Прима для знаходження мінімального кістяка\n"
        info += f"Параметри: n1={n1}, n2={n2}, n3={n3}, n4={n4}\n"
        info += f"n={n}, seed={seed}, k={k:.3f}\n"
        info += f"{' '*60}\n\n"

        info += f"Крок {self.step_count}:\n"
        info += f"Відвідані вершини: {sorted([v+1 for v in self.visited])}\n"
        info += f"Загальна вага MST: {self.total_weight}\n\n"

        if self.mst_edges:
            info += "Ребра в MST:\n"
            for i, (u, v, weight) in enumerate(self.mst_edges, 1):
                info += f" {i}. ({u+1}, {v+1}) - вага: {weight}\n"

```

```

else:
    info += "MST поки порожнє\n"

info += "\n"

if self.priority_queue and not self.algorithm_finished:
    info += "Доступні ребра (відсортовані за вагою):\n"
    for weight, u, v in self.priority_queue[:10]:
        info += f"  ({u+1}, {v+1}) - вага: {weight}\n"
    if len(self.priority_queue) > 10:
        info += f"  ... та ще {len(self.priority_queue) - 10} ребер\n"

info += f"\n{'='*60}\n"
info += "Матриця суміжності (Aundir):\n"
info += "  " + " ".join(f"{i+1:2}" for i in range(self.n)) + "\n"
for i in range(self.n):
    info += f"{i+1:2}: " + " ".join(f"{Aundir[i,j]:2}" for j in
range(self.n)) + "\n"

info += "\nМатриця ваг (W):\n"
info += "  " + " ".join(f"{i+1:3}" for i in range(self.n)) + "\n"
for i in range(self.n):
    info += f"{i+1:2}: " + " ".join(f"{int(W[i,j]):3}" for j in
range(self.n)) + "\n"

self.info_text.insert(tk.END, info)

if __name__ == "__main__":
    graph = Graph(Aundir, W)

    root = tk.Tk()
    root.title(f"Мінімальний кістяк графа - Алгоритм Прима (Варіант {seed})")
    root.geometry("800x900")

    app = PrimMSTApp(root, graph, W)

    print(f"\nGraph created with {graph.n} vertices")
    print(f"Total edges: {len(graph.get_all_edges())}")
    print(f"Starting Prim's algorithm from vertex 1...")

    root.mainloop()

```

Скріншоти роботи програми

```
Parameters: n1=4, n2=4, n3=1, n4=1  
n=11, seed=4411, k=0.935  
Algorithm: Prim (n4=1 is odd)
```

Undirected adjacency matrix Aundir:

```
[[0 1 1 1 0 1 1 1 1 1 0]  
 [1 0 1 0 0 1 1 1 1 0 0]  
 [1 1 0 1 0 0 1 1 0 0 1]  
 [1 0 1 0 1 0 1 1 1 0 1]  
 [0 0 0 1 0 1 0 0 1 0 0]  
 [1 1 0 0 1 0 1 1 0 0 1]  
 [1 1 1 1 0 1 0 0 1 1 0]  
 [1 1 1 1 0 1 0 0 1 1 1]  
 [1 1 0 1 1 0 1 1 0 1 0]  
 [1 0 0 0 0 0 1 1 1 0 0]  
 [0 0 1 1 0 1 0 1 0 0 0]]
```

Weight matrix W:

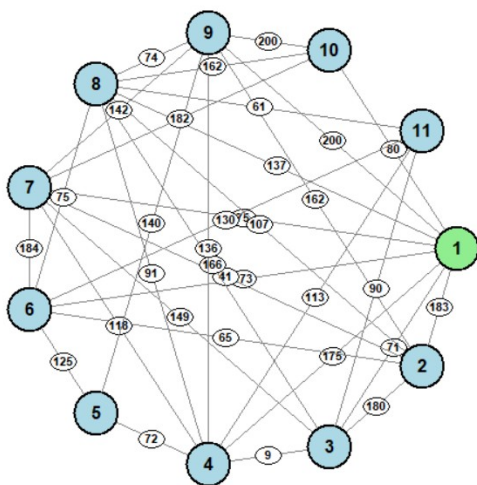
```
[[ 0. 183.  71. 175.   0. 173.  75. 137. 200.  80.   0.]  
 [183.   0. 180.   0.   0.  65.  41. 107. 162.   0.   0.]  
 [ 71. 180.   0.   9.   0.   0. 149. 166.   0.   0.  90.]  
 [175.   0.   9.   0.  72.   0. 118.  91. 136.   0. 113.]  
 [  0.   0.   0.  72.   0. 125.   0.   0. 140.   0.   0.]  
 [173.  65.   0.   0. 125.   0. 184.  75.   0.   0. 130.]  
 [ 75.  41. 149. 118.   0. 184.   0.   0. 142. 182.   0.]  
 [137. 107. 166.  91.   0.  75.   0.   0.  74. 162.  61.]  
 [200. 162.   0. 136. 140.   0. 142.  74.   0. 200.   0.]  
 [ 80.   0.   0.   0.   0.   0. 182. 162. 200.   0.   0.]  
 [  0.   0.  90. 113.   0. 130.   0.  61.   0.   0.   0.]]
```

Graph created with 11 vertices

Total edges: 33

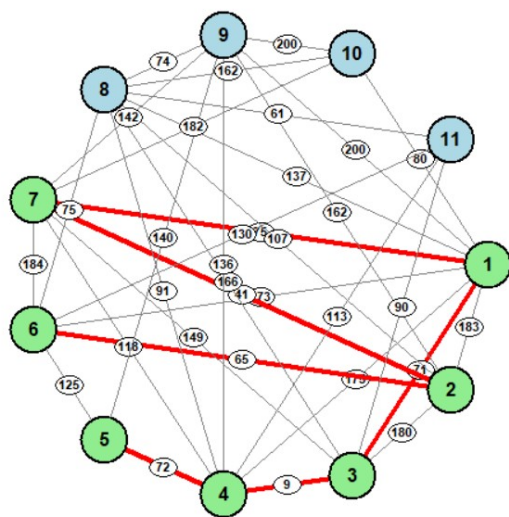
Starting Prim's algorithm from vertex 1...

□



Наступний крок

Скинути



Наступний крок

Скинути



Скинути

Висновок

Розроблена програма успішно реалізує генерацію неорієнтованого графа з 11 вершинами на основі випадкової матриці суміжності та ваг, які відповідають умовам варіанту. За допомогою графічного інтерфейсу на мові Python (бібліотека tkinter) реалізовано покрокове виконання алгоритму Прима для знаходження мінімального кістяка графа. Програма автоматично обчислює ваги ребер, будує остовне дерево, візуалізує вершини та ребра з урахуванням їх належності до MST, а також відображає змінні дані під час кожного кроку виконання. Інформаційна панель дозволяє відстежувати поточний стан алгоритму, наявні ребра у черзі пріоритетів та сумарну вагу дерева. В результаті користувач має можливість в інтерактивному режимі дослідити роботу алгоритму Прима на конкретному випадковому графі та зрозуміти процес побудови мінімального остовного дерева.